

DOCUMENTAÇÃO DO PROJETO 01

Desenvolvimento Web III

Aplicação Web com Node.js e HTTP Nativo

Integrante 1	Ryan Santos de Oliveira
Integrante 2	Eduardo Felipe dos Santos Alves
Curso / Disciplina	Desenvolvimento de Software Multiplataforma — Desenvolvimento Web III
Entrega	Projeto 01 — 26/08/2026 às 07h40

Documento elaborado com base no enunciado da atividade e na versão do projeto fornecida.

Sumário

1. Identificação do projeto
2. Objetivo
3. Descrição da solução
4. Tecnologias e recursos utilizados
5. Estrutura de arquivos
6. Estrutura das rotas
7. Funcionamento da aplicação
8. Principais trechos de código
9. Decisões de desenvolvimento
10. Tratamento de erros
11. Atendimento aos requisitos da atividade
12. Como executar o projeto
13. Considerações finais
14. Anexo — códigos-fonte

1. Identificação do projeto

O Projeto 01 foi desenvolvido para a disciplina de Desenvolvimento Web III, com o objetivo de aplicar conceitos de criação de servidores web, definição de rotas, leitura de arquivos e disponibilização de conteúdo utilizando Node.js.

Integrantes:

Ryan Santos de Oliveira

Eduardo Felipe dos Santos Alves

2. Objetivo

O objetivo da aplicação é apresentar os dois integrantes da equipe por meio de uma aplicação web servida por Node.js. O sistema disponibiliza uma página inicial, páginas individuais, currículos em PDF, imagens e uma resposta de erro para rotas inexistentes.

A solução foi construída sem o uso de frameworks como Express, utilizando diretamente o módulo HTTP nativo do Node.js. Dessa forma, o projeto permite demonstrar de maneira prática como requisições HTTP são recebidas, como as rotas são identificadas e como arquivos são enviados ao navegador.

3. Descrição da solução

A aplicação utiliza um servidor HTTP criado com o módulo nativo 'http' do Node.js. O arquivo server.js recebe cada requisição, analisa o caminho solicitado e direciona a resposta para o arquivo correspondente.

O conteúdo apresentado pelo sistema é composto principalmente por páginas HTML, arquivos PDF de currículo, imagens dos integrantes e uma página de erro 404. A leitura dos arquivos é centralizada na função readFile(), evitando repetição da lógica de acesso ao sistema de arquivos.

4. Tecnologias e recursos utilizados

Node.js — ambiente de execução do servidor.

Módulo http — criação do servidor HTTP sem framework.

Módulo url — análise do endereço solicitado.

Módulo fs — leitura dos arquivos armazenados no projeto.

Módulo path — construção segura dos caminhos dos arquivos.

HTML5 — criação das páginas apresentadas ao usuário.

PDF — disponibilização dos currículos dos integrantes.

JPEG — imagens dos integrantes.

Git/GitHub — forma de versionamento e entrega exigida pela atividade.

5. Estrutura de arquivos

A versão fornecida do projeto apresenta a seguinte estrutura:

```
Projeto_DSW/  
  ■■■ index.html  
  ■■■ server.js  
  ■■■ erro404.html  
  ■■■ Eduardo/  
    ■ ■■■ Sobre1.html  
    ■ ■■■ curriculo-2.pdf  
    ■ ■■■ Eduardo.jpeg  
  ■■■ Ryan/  
    ■■■ Sobre.html  
    ■■■ Curriculo-1.pdf  
    ■■■ Ryan.jpeg
```

A organização separa os arquivos de cada integrante em suas respectivas pastas, facilitando a manutenção e a identificação dos recursos utilizados por cada página.

6. Estrutura das rotas

Rota	Recurso	Função
/	index.html	Página inicial da aplicação.
/index	index.html	Acesso alternativo à página inicial.
/Eduardo/Sobre1	Eduardo/Sobre1.html	Página de apresentação do Eduardo.

Rota	Recurso	Função
/Eduardo/curriculo-2	Eduardo/curriculo-2.pdf	Currículo do Eduardo em PDF.
/Eduardo/imagem/Eduardo	Eduardo/Eduardo.jpeg	Imagem do Eduardo.
/Ryan/Sobre	Ryan/Sobre.html	Página de apresentação do Ryan.
/Ryan/Curriculo-1	Ryan/Curriculo-1.pdf	Currículo do Ryan em PDF.
/Ryan/imagem/Ryan	Ryan/Ryan.jpeg	Imagem do Ryan.

Observação importante: a estrutura implementada na versão entregue é funcional, porém difere parcialmente do exemplo sugerido no enunciado. O exemplo propõe, por exemplo, rotas separadas para '/aluno/sobre' e '/aluno/curriculo' e exige a rota '/projeto'. Na versão analisada, o currículo é servido diretamente por uma rota específica e a rota '/projeto' ainda não está implementada.

7. Funcionamento da aplicação

O usuário acessa `http://localhost:3000/`.

O servidor Node.js recebe a requisição HTTP.

O código verifica o caminho solicitado por meio de `url.parse()`.

Quando a rota corresponde a um recurso conhecido, o servidor define o Content-Type adequado.

A função `readFile()` localiza o arquivo no diretório do projeto e realiza sua leitura com `fs.readFile()`.

O conteúdo do arquivo é enviado na resposta HTTP.

Caso a rota não exista, o servidor retorna o status HTTP 404 e apresenta a página `erro404.html`.

8. Principais trechos de código

8.1 Importação dos módulos

```
const url = require('url');
const http = require('http');
const fs = require('fs');
const path = require('path');
```

Os módulos nativos utilizados fornecem as funcionalidades necessárias para criar o servidor, interpretar URLs, ler arquivos e montar os caminhos utilizados pelo sistema.

8.2 Função de leitura de arquivos

```
function readFile(response, filePath){
  const absolutePath = path.join(__dirname, filePath);
  console.log("Tentando ler o arquivo em:", absolutePath);
  fs.readFile(absolutePath, function(err, data){
    if (err) {
      console.error("ERRO DETALHADO:", err.message);
      response.writeHead(404, {"Content-Type": "text/html; charset=utf-8"});
      response.end(`<h1>Erro 404: Arquivo não encontrado</h1>
<p>0 Node tentou procurar em: <code>${absolutePath}</code></p>`);
      return;
    }
    response.end(data);
  });
}
```

Esse trecho centraliza a leitura dos arquivos. O `path.join()` combina o diretório do projeto com o caminho relativo do recurso. Em seguida, `fs.readFile()` realiza a leitura. Se ocorrer um erro, uma resposta 404 é enviada.

8.3 Identificação das rotas

```
var parts = url.parse(request.url);
if(parts.path === "/" || parts.path === "/index"){
  response.writeHead(200, {"Content-Type": "text/html"});
  readFile(response, 'index.html');
}
else if(parts.path === "/Eduardo/Sobre1"){
```

```
response.writeHead(200, {"Content-Type": "text/html"});
readFile(response, 'Eduardo/Sobre1.html');
}
```

A estrutura condicional compara o caminho solicitado com as rotas previamente definidas. Para cada rota, o servidor informa o tipo de conteúdo e chama `readFile()` com o arquivo correspondente.

8.4 Servidor e porta

```
var server = http.createServer(callback);
server.listen(3000);
console.log("Servidor Iniciado em http://localhost:3000/");
```

O `createServer()` cria o servidor HTTP utilizando a função `callback` como responsável pelo processamento das requisições. O método `listen(3000)` coloca a aplicação em funcionamento na porta 3000.

9. Decisões de desenvolvimento

Uso do Node.js sem Express: a escolha permite demonstrar diretamente os conceitos fundamentais de HTTP e roteamento trabalhados em aula.

Separação dos arquivos por integrante: cada aluno possui sua própria pasta, contendo sua página, currículo e imagem.

Função `readFile()`: a lógica de leitura foi reutilizada para diferentes tipos de arquivos, reduzindo duplicação de código.

Definição explícita de Content-Type: HTML, PDF e JPEG recebem tipos de conteúdo apropriados para que o navegador interprete os arquivos corretamente.

Página de erro: rotas inexistentes são tratadas com status 404 e uma página específica.

Navegação por links: as páginas utilizam links HTML para conectar a página inicial às páginas individuais e aos currículos.

10. Tratamento de erros

O projeto possui dois níveis básicos de tratamento. Primeiro, quando uma rota não corresponde a nenhuma condição do servidor, é retornado o status HTTP 404 e o arquivo `erro404.html` é carregado. Segundo, quando um arquivo solicitado não pode ser lido pelo sistema de arquivos, a função `readFile()` registra o erro no terminal e retorna uma resposta 404.

```
else {
  response.writeHead(404, {"Content-Type": "text/html"});
  readFile(response, 'erro404.html');
}
```

11. Atendimento aos requisitos da atividade

A tabela abaixo relaciona o enunciado com a implementação encontrada na versão do projeto analisada.

Requisito	Situação	Observação
Rota principal /	Atendido	Abre <code>index.html</code> .
Nome dos dois integrantes	Atendido	Apresentados na página inicial.
Apresentação e instruções	Atendido	Existem na página inicial.
Links para páginas individuais	Atendido	Existem links para Eduardo e Ryan.
Página individual de cada aluno	Atendido	Cada integrante possui uma página HTML.
Currículo em PDF	Atendido	Cada integrante possui um PDF acessível por rota.
Rota /projeto	Pendente	Não foi encontrada na versão fornecida.

Requisito	Situação	Observação
Documentação disponibilizada pela aplicação	Pendente	O documento desta entrega deve ser convertido para PDF e disponibilizado pela rota /projeto.
Todos os códigos-fonte na documentação	Atendido neste documento	Incluídos no Anexo.
Explicação dos principais códigos	Atendido	Apresentada na seção 8.
Considerações finais	Atendido	Apresentadas na seção 13.

Recomendação antes da entrega: implementar a rota /projeto no server.js e disponibilizar a versão PDF desta documentação nessa rota, pois esse é um requisito obrigatório do enunciado.

12. Como executar o projeto

Instalar o Node.js no computador.

Abrir um terminal na pasta Projeto_DSW.

Executar o comando: node server.js

Aguardar a mensagem indicando que o servidor foi iniciado.

Abrir no navegador: http://localhost:3000/

Testar as rotas de apresentação e os links dos currículos.

Para encerrar o servidor, pressione Ctrl + C no terminal.

13. Considerações finais

O projeto permitiu aplicar conceitos fundamentais de desenvolvimento web utilizando Node.js, especialmente a criação de um servidor HTTP, o tratamento de requisições, a definição de rotas e a entrega de diferentes tipos de arquivos.

A organização por pastas facilita a identificação dos recursos de cada integrante, enquanto a função readFile() concentra a lógica de leitura de arquivos. A solução também demonstra o uso de códigos de status HTTP e de cabeçalhos Content-Type para a correta entrega de HTML, PDF e imagens.

Como melhoria necessária para atender integralmente ao enunciado, recomenda-se acrescentar a rota /projeto e disponibilizar nela a documentação em PDF. Também é possível evoluir a estrutura das rotas individuais para separar explicitamente as páginas de apresentação e currículo, caso essa organização seja desejada pela dupla.

14. Anexo — códigos-fonte

A seguir estão os códigos-fonte encontrados na versão fornecida do projeto. Os trechos são apresentados para permitir a consulta da implementação e atender ao requisito de documentação dos códigos utilizados.

14.1 server.js

```
/* Fatec 217 - Aula 19/08/2026 - 3 Sem - DSM
Nome: Ryan Santos - ryan.s.oliveira09@gmail.com
Descricao primeiro programa de node.js com webserver (sem framework).
Objetivo ter o primeiro contato com node.js e webserver.
Versao 04: Abrir arquivos no end point.
*/
// Carregar os modulos
const url = require('url');
const http = require('http');
const fs = require('fs');
const path = require('path');
// Funcao para ler arquivo e enviar no http:
function readFile(response, filePath){
const absolutePath = path.join(__dirname, filePath);
```

```
// Mostra no CMD qual caminho o Node está tentando ler no seu HD
console.log("Tentando ler o arquivo em:", absolutePath);
fs.readFile(absolutePath, function(err, data){
  if (err) {
    console.error("ERRO DETALHADO:", err.message);
    response.writeHead(404, {"Content-Type": "text/html; charset=utf-8"});
    response.end(`<h1>Erro 404: Arquivo não encontrado</h1><p>0 Node tentou procurar em: <code>${absolutePath}</code></p>`);
    return;
  }
  response.end(data);
});
}

// Funcao Callback para utilizar no server http:
var callback = function(request, response){
  // Faz o parse da URL, separando o caminho (end-points)
  var parts = url.parse(request.url);
  // Ajuste dos end-points e direcionamento para os caminhos das pastas corretas:
  if(parts.path === "/" || parts.path === "/index"){
    response.writeHead(200, {"Content-Type": "text/html"});
    readFile(response, 'index.html');
  }
  else if(parts.path === "/Eduardo/Sobre1"){
    response.writeHead(200, {"Content-Type": "text/html"});
    readFile(response, 'Eduardo/Sobre1.html'); // Aponta para a pasta Eduardo/
  }
  else if(parts.path === "/Eduardo/curriculo-2"){
    response.writeHead(200, {"Content-Type": "application/pdf"});
    readFile(response, 'Eduardo/curriculo-2.pdf');
  }
  else if(parts.path === '/Eduardo/imagem/Eduardo'){
    response.writeHead(200, {"Content-Type": "image/jpeg"});
    readFile(response, 'Eduardo/Eduardo.jpeg');
  }
  else if(parts.path === "/Ryan/Sobre"){
    response.writeHead(200, {"Content-Type": "text/html"});
    readFile(response, 'Ryan/Sobre.html'); // Aponta para a pasta Ryan/
  }
  else if(parts.path === "/Ryan/Curriculo-1"){
    response.writeHead(200, {"Content-Type": "application/pdf"});
    readFile(response, 'Ryan/Curriculo-1.pdf');
  }
  else if(parts.path === '/Ryan/imagem/Ryan'){
    response.writeHead(200, {"Content-Type": "image/jpeg"});
    readFile(response, 'Ryan/Ryan.jpeg');
  }
  else {
    response.writeHead(404, {"Content-Type": "text/html"});
    readFile(response, 'erro404.html');
  }
};

// Criação e configuração de um Servidor http:
var server = http.createServer(callback);
server.listen(3000);
console.log("Servidor Iniciado em http://localhost:3000/");
```

14.2 index.html

```
<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<title>Página Inicial - Projeto DSM</title>
</head>
<body>
<h1>Projeto DSM - Disciplina de Desenvolvimento Web 3 - Webserver em Node.js</h1>
<section>
<h2>Apresentação do Projeto</h2>
<p>Este projeto consiste em um servidor web desenvolvido em Node.js (sem a utilização de frameworks como o Express) para a disciplina de Desenvolvimento de Software Multiplataforma (DSM) da Fatec. O objetivo principal é explorar o funcionamento de rotas, manipulação de arquivos estáticos (HTML, PDF e imagens) e respostas HTTP nativas do Node.js.</p>
</section>
<hr>
<section>
<h2>Instruções de Navegação</h2>
<p>Para navegar pelo sistema, utilize os botões abaixo:</p>
<ul>
<li>Clique no botão referente ao integrante desejado para visualizar sua página <strong>"Sobre"</strong>.</li>
<li>Dentro de cada página individual, você encontrará as informações do integrante, o acesso ao seu <strong>Currículo (PDF)</strong> e um botão para retornar a esta tela inicial.</li>
</ul>
</section>
<hr>
<section>
<h2>Perfis dos Alunos</h2>
<h3>Eduardo Felipe dos Santos Alves</h3>
<a href="/Eduardo/Sobre1">
```

```

<button>Sobre Eduardo</button>
</a>
<br><br>
<h3>Ryan tanananan
</h3>
<a href="/Ryan/Sobre">
<button>Sobre Ryan</button>
</a>
</section>
</body>
</html>

```

14.3 Eduardo/Sobre1.html

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<title>Sobre Eduardo</title>
</head>
<body>
<h1>Página Sobre - Eduardo</h1>
<p>Bem-vindo à minha página!</p>
<h1>Olá sou o eduardo alves tenho 20 anos e nesse momento estou cursando Desenvolvimento de Software Multiplataforma na FATEC Diadema </h1>
<h1> Estou realizando o começo de um projetinho para conseguir entender melhor como funciona se criarmos um servidor com algumas informações </h1>
<h1>Agora nesse momento vou criar uma index linkando com uma página com minhas informações e com as informações do Ryan</h1>
<a href="/Eduardo/curriculo-2" target="_blank">
<button>Visualizar Currículo (PDF)</button>
</a>
<br><br>
<a href="/index">
<button>Voltar para o Início</button>
</a>
</body>
</html>

```

14.4 Ryan/Sobre.html

```

<!DOCTYPE html>
<html lang="pt-BR">
<head>
<meta charset="UTF-8">
<title>Sobre Ryan</title>
</head>
<body>
<h1> Ryan Santos de Oliveira</h1>
<h2> Tenho 19 anos, estou no terceiro semestre do curso: Desenvolvimento de Software Multiplataforma. </h2>
<br>
<h2> Este é meu primeiro servidor com node.js </h2>
<a href="/Ryan/Curriculo-1" target="_blank">
<button>Visualizar Currículo (PDF)</button>
</a>
<br><br>
<a href="/index">
<button>Voltar para o Início</button>
</a>
</body>
</html>

```

14.5 erro404.html

```

<h1> Erro </h1>
<h2> Por gentileza, saia da pagina.</h2>

```